

Previously, we discussed the motivation of abstraction and the science of them. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat. In this article, we present a more concrete motivation. An application of abstraction in application development.¹

A conventional view of application development would be that the client and the server form an architecture where the server is the orchestrator and the client merely fetches and presents. A full-stack developer, however, would say that this view of the application is not necessary: the programming of the application is much more abstract and high-level and is often quite syntactically different from the implementation code, and that this difference should be reduced. You may perform a fetch to tier 2—which fetches tier 3—to get some data then deserializes it; as visualized by a sequence diagram. But your brain may see it as a single orchestration where data flows from tier 3 to a state variable in tier 1. At the core of this system is the concept of *flow*—not the sort of instructions that one might build into a machine or write a computer program, but the grammar to describe and declare between humans.

For example, a Reason code may orchestrate that data (pull request items filtered by target-post-id) shall flow from a source to the current application. It does so by opening a *portal*—written as a Reason functor—into an abstract data repository then defines a flow, like so

```
module At_repo0(S : {
  include Entities.S
  let tgt_post_id : string })
{ open S
  let rec q = () =>
    foreach(prs') @@ o =>
      where(Pull_request.post_id(o) =@
str(tgt_post_id)) @@ () =>
```

¹currently, it is at POC stage, it is a groundwork where more can be developed.

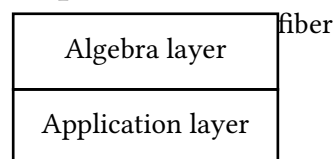
```
yield(o)
and prs' = () => table @@
("pullrequest", prs())
}
```

and this orchestration is then used, Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore.

```
[@react.component]
let make () => {
  ...
  let open Remote(module Env);
  watch0(() => Promise.Syntax.{
    let* prs = Buf_read.of_flow(module
At_repo0);
    ...
  }));
}
```

This orchestration syntax is very expressive, it resembles a direct relational data query, the module *S* in the functor *At_repo0* provides all entities and relationships symbol vocabulary in the bkhack system that a programmer can use. This is an abstraction. In the implementation layer, each *At_repo0*, upon plug-in to a *Buf-read.of-flow*, compiles to specialized and secured fetch code. This has been one example, being data repository portal². Other kinds include stylesheet portal, pagegen portal, fiber [1], and server-side react components.

Implementation



First-class full-stack build process

Figure 1

Figure 2

²currently, a POC implementation of this exists, called *free sql*

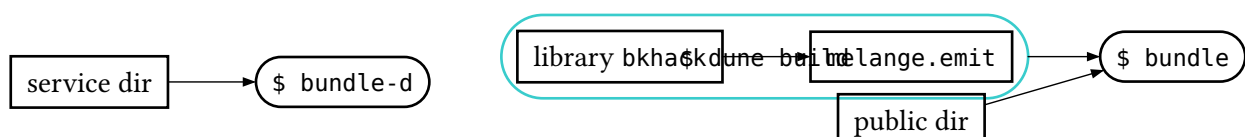


Figure 1: Version 1

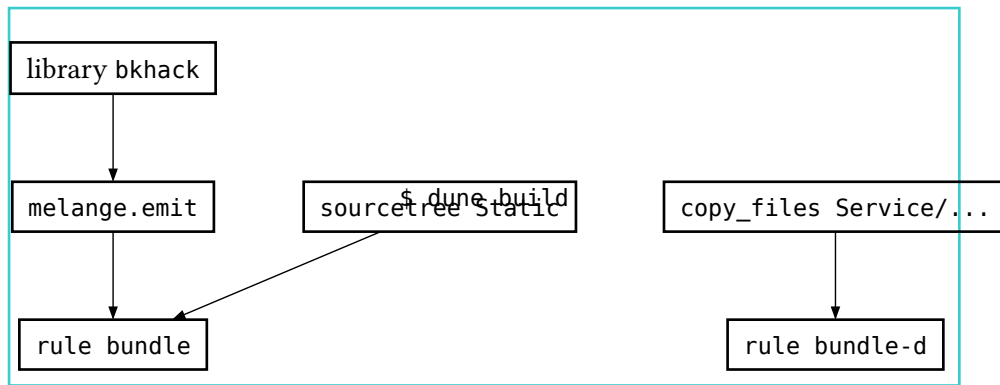


Figure 2: Version 2

Design with escape-hatches in mind, self-recovery

Even with the goal of a full-stack framework in mind, we must admit the reality that it is hard and is an contextual evolving process to design an abstraction, we embrace it that our system is a living evolving one. And the view of a full-stack abstraction is not always good, sometimes it's good to have clear separation of responsibilities. And the goal isn't uniformly shared by all developers.

Hence, while the core app is a Reason front-end bundle, there are other aspects of the app. The styling of the app is shifted towards the static front-end bundle. The serving of data is shifted towards the service bundle of the app. The final-state, ideal of the app is one where only the core remains. The current reality is that there are multiple aspects / bundles of the app, and we try to shrink these aspects / bundles as much as we could, while upholding functional and non-functional requirements.

APIs will be eventually converted from references and fetches to embeddings and abstract algebras. In the case when conversion is difficult, or when there is disagreement, we will be fine with the way things are. In other words, when the full-stack abstraction fails, it can self-recover by defaulting to a reliable raw, concrete implementation.

Heterogeneous

Heterogeneity, specifically *implicit heterogeneity*, as an aspect of a full-stack system, is the idea that even when the full-stack system is programmable in one homogeneous unit, its output doesn't have to be homogeneous but instead can be heterge-

neous.

The bkhack system, albeit full-stack, is being split into a front-end part and an optional back-end part. Within this front-end part, the Reason code is one homogeneous source-tree, but is actually being implicitly split into multiple page outputs (multi-page application) as well as being generative of stylesheet (cssgen) and page layout (pagegen) static asset.

This implicit heterogeneity ensures *zero-cost abstraction*.

This heterogeneity is made possible because our system is built before deployment—it is staged and has layers.

Related works

Ruby on Rails, Expo, Phoenix, Eliom

- [1] in *Abstracting in-browser parallelism and more with fibers*, in bkhack developer handbook., TTB-HCMUT. [Online]. Available: <https://github.com/ttb-hcmut/bkhack/blob/main/src/Devbook/output/page-fiber.pdf>